# Build a Smart Real Estate Listings Platform

## 🧠 Business Context

Real estate marketplaces look simple on the surface, but in reality, they struggle with:

- messy, inconsistent data,
- duplicated or incomplete offers,
- vague or misleading descriptions,
- users searching with *intent*, not exact filters.

A modern platform is not just a database of flats.

It must:

- ingest real-world data from external sources,
- normalise and structure it,
- allow users to explore listings efficiently,
- and present offers clearly and consistently.

AI can help both **during development** and **inside the product** - from parsing unstructured content to improving search relevance and presentation.

---

## 🎯 Your Mission

Build a **simplified real estate listings platform** inspired by services like *otodom.pl*.

The system should:

### 1️⃣ Acquire data

- Crawl or fetch random **~100 real estate offers** from *any* publicly available marketplace (e.g. *otodom*, *tabelaofert*, *sprzedajemy*, *okolica*, or another you find easiest).

- Extract meaningful fields from the raw content
  (You decide what is meaningful 🙂)

### 2️⃣ Store & normalize data

- Save offers in a database.
- Handle inconsistencies, missing values, or noisy descriptions in a reasonable way.

### 3️⃣ Expose listings

- A listing page with:
    - basic search (text + filters of your choice),
    - pagination or lazy loading.
- An offer details page with full information.

### 4️⃣ Use AI intentionally (not required)

- Use AI where it *adds value*, for example:
    - parsing or structuring scraped content,
    - generating summaries,
    - improving search,
    - deduplicating or enriching data.
- Or nothing. :) You choose how deeply the AI is involved.

### 5️⃣ Deliver a clean MVP

- Clear structure,
- readable code,
- sensible UX (no design perfection required).

---

## 🧵 Technical Constraints

- **Backend:** TS + Node.JS (but you can use your language of choice if explained why)
- **Frontend:** up to you
- **Database:** MySQL
- **AI:** free choice (API, local model, prompt-based, etc.)

**You decide:**

- crawling approach,
- data model,
- AI integration,
- search strategy,
- tradeoffs.

---

## 📦 Deliverables

### 1️⃣ Short reasoning document (1 pager)

This is the **most important part**.

Include:

- What data did you decide to extract, and why
- How you handled unstructured or low-quality data
- Where and why you used AI
- One key assumption you made
- One success metric for the product
- One failure mode or limitation of your approach
- What would you improve with more time

---

### 2️⃣ Working prototype

- Ability to:
    - browse listings,

    - search,
    - open an offer page

Mocking some parts is acceptable if clearly explained.

---

### 3️⃣ Example scenarios

Provide **two short example user journeys**, e.g.:

- Example A:
  The user searches for flats in Kraków with 40-80 m2 and finds a relevant one.

- [Level: Hard, optional] Example B:
  We show a chat box instead of a search input. The user searches with a vague intent and relies on your data normalisation/AI logic to get good results, e.g. *"I want a nice, cheap flat, 40m in Kraków"*

---

## 🥷 Important Notes

- This is **not** a scraping challenge - correctness and reasoning matter more than scale.
- The UI does not need to be pretty, but it should be clean and usable.
- Avoid overengineering; focus on a solid MVP.
- We care more about *how you think* than how many features you ship.
- Coding with AI is more than appreciated.
- Do not spend more than 6 hours :)